1. **Create a Simple Pod Using YAML**
   Task: Write a YAML file to create a Pod named firstpod with an nginx container. Verify the Pod creation using kubectl get pods and check the logs of the container using kubectl logs firstpod.

Objective

To create a simple Nginx Pod using a Kubernetes YAML manifest file.

Step 1: Create a YAML file

vi task1-firstpod.yml

Step 2: Add the following configuration

```
apiVersion: v1
kind: Pod
metadata:
 name: firstpod
 labels:
  app: nginx
spec:
 containers:
  - name: nginx-container
    image: nginx:latest
    ports:
     - containerPort: 80
```

Step 3: Deploy the Pod

kubectl apply -f task1-firstpod.yml

Expected output:

pod/firstpod created

Step 4: Verify the Pod

kubectl get pods

To get detailed information:

kubectl describe pod firstpod

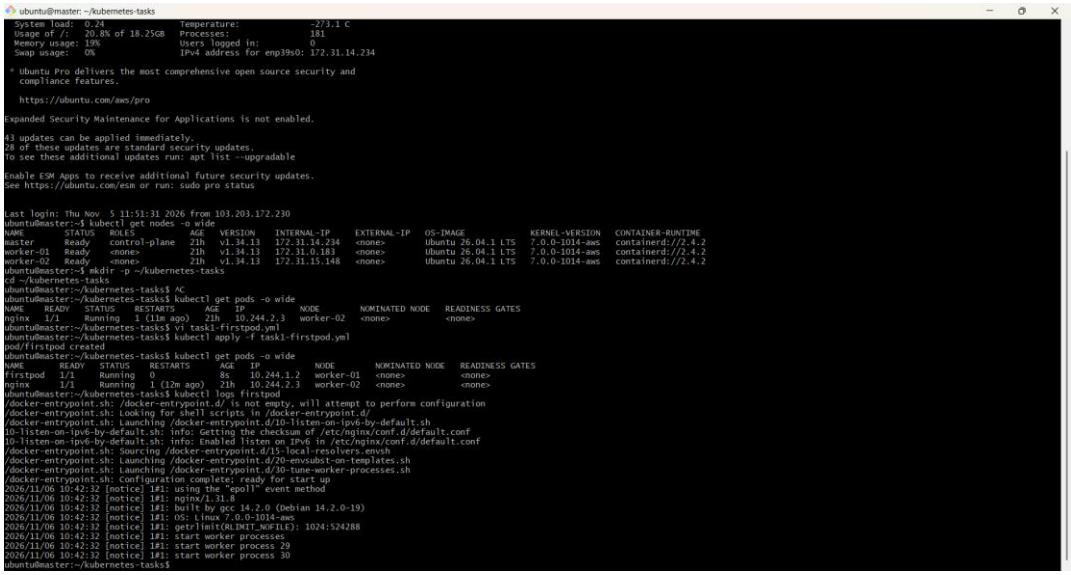
Expected result:

- Pod name: firstpod
- Image: nginx:latest
- Status: Running
- Container port: 80

Explanation

A Pod is the smallest deployable unit in Kubernetes. It can contain one or more containers that share networking and storage resources.

In this task, we created a Pod containing one Nginx container.



```
ubuntu@master: ~/kubernetes-tasks
  System load:  0.24                Temperature:                -273.1 C
  Usage of /:   20.8% of 18.25GB    Processes:                  181
  Memory usage: 19%                 Users logged in:            0
  Swap usage:   0%                  IPv4 address for enp39s0: 172.31.14.234

 * Ubuntu Pro delivers the most comprehensive open source security and
   compliance features.

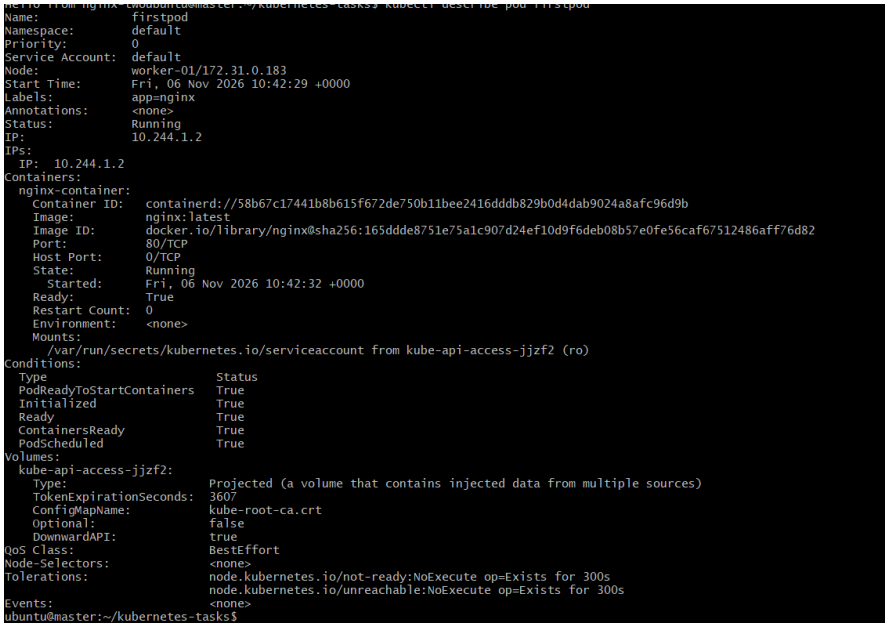
   https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.

43 updates can be applied immediately.
28 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


Last login: Thu Nov  5 11:51:31 2026 from 103.203.172.230
ubuntu@master:~$ kubectl get nodes -o wide
NAME        STATUS   ROLES           AGE   VERSION   INTERNAL-IP     EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION   CONTAINER-RUNTIME
master      Ready    control-plane   21h   v1.34.13  172.31.14.234   <none>        Ubuntu 26.04.1 LTS   7.0.0-1014-aws   containerd://2.4.2
worker-01   Ready    <none>          21h   v1.34.13  172.31.0.183    <none>        Ubuntu 26.04.1 LTS   7.0.0-1014-aws   containerd://2.4.2
worker-02   Ready    <none>          21h   v1.34.13  172.31.15.148   <none>        Ubuntu 26.04.1 LTS   7.0.0-1014-aws   containerd://2.4.2
ubuntu@master:~$ mkdir -p ~/kubernetes-tasks
cd ~/kubernetes-tasks
ubuntu@master:~/kubernetes-tasks$ AC
ubuntu@master:~/kubernetes-tasks$ kubectl get pods -o wide
NAME    READY   STATUS    RESTARTS      AGE   IP           NODE        NOMINATED NODE   READINESS GATES
nginx   1/1     Running   1 (11m ago)   21h   10.244.2.3   worker-02   <none>           <none>
ubuntu@master:~/kubernetes-tasks$ vi task1-firstpod.yml
ubuntu@master:~/kubernetes-tasks$ kubectl apply -f task1-firstpod.yml
pod/firstpod created
ubuntu@master:~/kubernetes-tasks$ kubectl get pods -o wide
NAME       READY   STATUS    RESTARTS      AGE   IP           NODE        NOMINATED NODE   READINESS GATES
firstpod   1/1     Running   0             8s    10.244.1.2   worker-01   <none>           <none>
nginx      1/1     Running   1 (12m ago)   21h   10.244.2.3   worker-02   <none>           <none>
ubuntu@master:~/kubernetes-tasks$ kubectl logs firstpod
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/11/06 10:42:32 [notice] 1#1: using the "epoll" event method
2026/11/06 10:42:32 [notice] 1#1: nginx/1.31.8
2026/11/06 10:42:32 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/11/06 10:42:32 [notice] 1#1: OS: Linux 7.0.0-1014-aws
2026/11/06 10:42:32 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:524288
2026/11/06 10:42:32 [notice] 1#1: start worker processes
2026/11/06 10:42:32 [notice] 1#1: start worker process 29
2026/11/06 10:42:32 [notice] 1#1: start worker process 30
ubuntu@master:~/kubernetes-tasks$
```



```
Name:             firstpod
Namespace:        default
Priority:         0
Service Account:  default
Node:             worker-01/172.31.0.183
Start Time:       Fri, 06 Nov 2026 10:42:29 +0000
Labels:           app=nginx
Annotations:      <none>
Status:           Running
IP:               10.244.1.2
IPs:
  IP:  10.244.1.2
Containers:
  nginx-container:
    Container ID:   containerd://58b67c17441b8b615f672de750b11bee2416dddb829b0d4dab9024a8afc96d9b
    Image:          nginx:latest
    Image ID:       docker.io/library/nginx@sha256:165ddde8751e75a1c907d24ef10d9f6deb08b57e0fe56caf67512486aff76d82
    Port:           80/TCP
    Host Port:      0/TCP
    State:          Running
      Started:      Fri, 06 Nov 2026 10:42:32 +0000
    Ready:          True
    Restart Count:  0
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-jjzf2 (ro)
Conditions:
  Type                        Status
  PodReadyToStartContainers   True
  Initialized                 True
  Ready                       True
  ContainersReady             True
  PodScheduled                True
Volumes:
  kube-api-access-jjzf2:
    Type:                     Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:   3607
    ConfigMapName:            kube-root-ca.crt
    Optional:                 false
    DownwardAPI:              true
QoS Class:                    BestEffort
Node-Selectors:               <none>
Tolerations:                  node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                              node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:                       <none>
ubuntu@master:~/kubernetes-tasks$
```

2. **Set Environment Variables in a Pod**
   Task: Modify the YAML file to include environment variables myname: sabair and City: Hyderabad. Deploy the Pod and use kubectl exec <pod_name> -- env to check if the environment variables are set properly.

Objective

To create a Pod and pass environment variables to its container.

Step 1: Create a YAML file

vi task2-env-pod.yml

Step 2: Add the configuration

```
apiVersion: v1
kind: Pod
metadata:
 name: env-pod
spec:
 containers:
  - name: nginx-container
    image: nginx:latest
    env:
     - name: myname
       value: "sabair"
     - name: City
       value: "Hyderabad"
```

Step 3: Deploy the Pod

kubectl apply -f task2-env-pod.yml

Step 4: Verify the Pod

kubectl get pods

Step 5: Verify environment variables

Check the first variable:

kubectl exec env-pod -- printenv myname

Output:

sabair

Check the second variable:

kubectl exec env-pod -- printenv City

Output:

Hyderabad

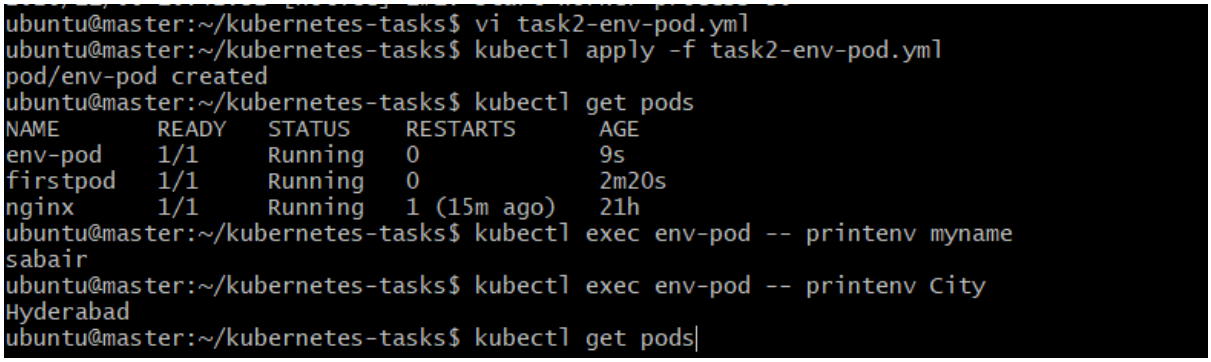
Explanation

Environment variables are used to pass configuration values to containers.

In this task, we passed two environment variables:

- myname with value sabair.
- City with value Hyderabad.

The env section is used to define environment variables inside a container.



3. **Deploy a Pod with Commands (Args) in YAML**
   Task: Modify the YAML file to add args that instruct the container to sleep for 50 seconds. Deploy the Pod and use kubectl describe pod to verify the args are correctly passed to the container.

Objective

To execute custom commands inside a container using the command and args fields.

The container should:

1. Print a starting message.
2. Sleep for 50 seconds.
3. Print a completion message.
4. Exit successfully.

Step 1: Create a YAML file

vi task3-sleep-pod.yml

Step 2: Add the configuration

```
apiVersion: v1
kind: Pod
metadata:
 name: sleep-pod
spec:
 restartPolicy: Never
 containers:
  - name: sleep-container
    image: nginx:latest
    command: ["sh", "-c"]
    args:
     - "echo 'Container started'; sleep 50; echo 'Container completed'"
```

Step 3: Deploy the Pod

kubectl apply -f task3-sleep-pod.yml

Step 4: Verify the Pod status

kubectl get pods

Initially, the Pod shows:

sleep-pod 0/1 Running

After 50 seconds, it shows:

sleep-pod 0/1 Completed

Step 5: Check container logs

kubectl logs sleep-pod

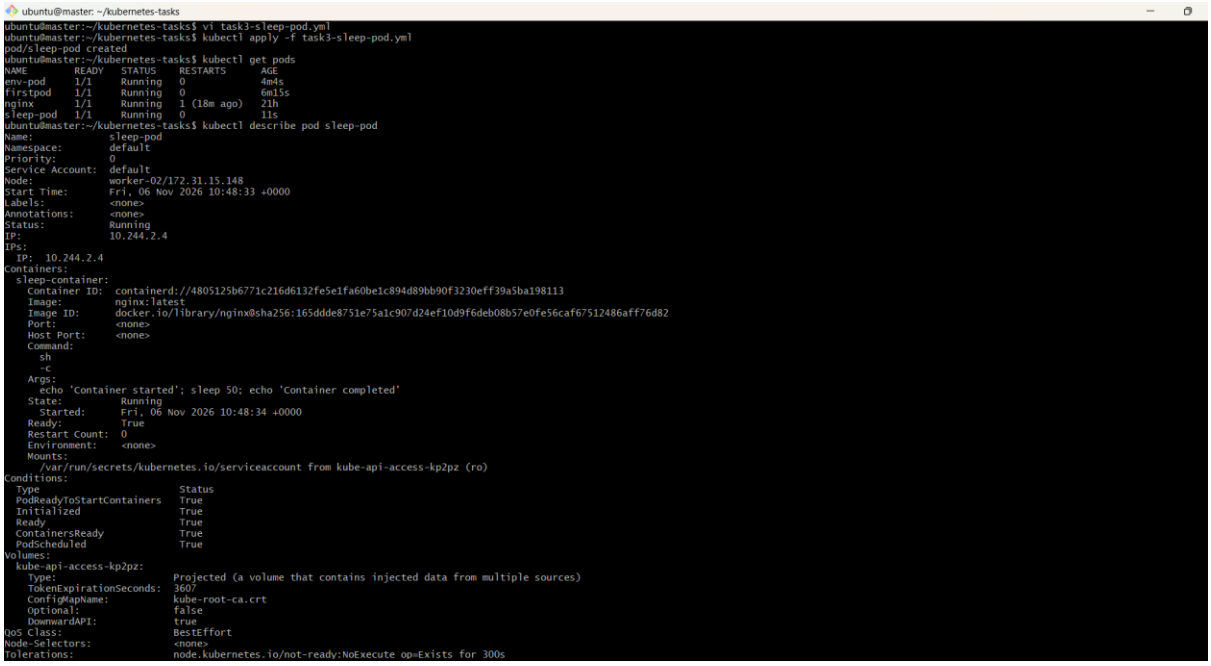
Output:

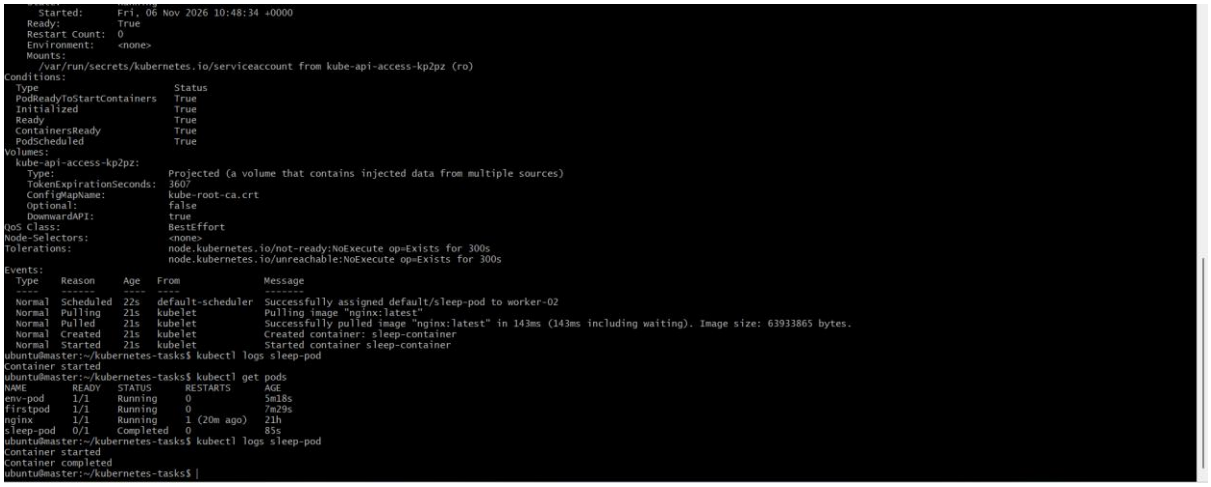
Container started

Container completed

Explanation

- command specifies the executable command to run inside the container.
- args provides arguments to that command.
- sleep 50 keeps the process running for 50 seconds.
- restartPolicy: Never prevents Kubernetes from restarting the container after successful completion.





4. **Create a Pod with Two Containers**
   Task: Create a YAML file to define a Pod with two nginx containers inside. Use kubectl exec to access both containers and verify that both containers can communicate through the same network (e.g., using telnet between them).

Objective

To create a single Pod containing two Nginx containers and verify communication between them.

Both containers share the same Pod network namespace and can communicate using localhost.

Since both Nginx containers cannot listen on the same port, the second container is configured to listen on port 8080.

Step 1: Create a YAML file

vi task4-two-containers.yml

Step 2: Add the configuration

```
apiVersion: v1
kind: Pod
metadata:
 name: two-container-pod
spec:
 containers:
  - name: nginx-one
    image: nginx:alpine
    ports:
     - containerPort: 80

  - name: nginx-two
    image: nginx:alpine
    command: ["/bin/sh", "-c"]
    args:
     - |
       cat > /etc/nginx/conf.d/default.conf <<'EOF'
       server {
          listen 8080;
          location / {
             return 200 "Hello from nginx-two on port 8080";
          }
       }
```

```
      EOF
      nginx -g 'daemon off;'
    ports:
     - containerPort: 8080
```

Step 3: Deploy the Pod

kubectl apply -f task4-two-containers.yml

Step 4: Verify the Pod

kubectl get pods

Expected output:

two-container-pod 2/2 Running

The 2/2 indicates that both containers are running and ready.

Step 5: Test communication from nginx-one to nginx-two

kubectl exec two-container-pod -c nginx-one -- wget -qO- http://127.0.0.1:8080

Expected output:

Hello from nginx-two on port 8080

Step 6: Test communication from nginx-two to nginx-one

kubectl exec two-container-pod -c nginx-two -- wget -qO- http://127.0.0.1:80

Expected result: Nginx welcome page HTML.

Explanation

Containers inside the same Pod share:

- The same network namespace.
- The same Pod IP address.
- The same localhost interface.
- Configured shared volumes, when provided.

They can communicate with each other using localhost and different listening ports.

```
ubuntu@master:~/kubernetes-tasks$ vi task4-two-containers.yml
ubuntu@master:~/kubernetes-tasks$ kubectl apply -f task4-two-containers.yml
pod/two-container-pod created
ubuntu@master:~/kubernetes-tasks$ kubectl get pods
NAME              READY   STATUS      RESTARTS      AGE
env-pod           1/1     Running     0             24m
firstpod          1/1     Running     0             27m
nginx             1/1     Running     1 (39m ago)   21h
sleep-pod         0/1     Completed   0             21m
two-container-pod   2/2     Running     0             7s
ubuntu@master:~/kubernetes-tasks$ kubectl describe pod two-container-pod
Name:             two-container-pod
Namespace:        default
Priority:         0
Service Account:  default
Node:             worker-02/172.31.15.148
Start Time:       Fri, 06 Nov 2026 11:09:30 +0000
Labels:           <none>
Annotations:      <none>
Status:           Running
IP:               10.244.2.5
IPs:
  IP:  10.244.2.5
Containers:
  nginx-one:
    Container ID:   containerd://a8acba1766f853f945188dde7d8aa3e2a2727f081b3a2050eba2d81c6d5c4e18
    Image:          nginx:alpine
    Image ID:       docker.io/library/nginx@sha256:713fc2ebfd92e8217db451477182ee91944b123d2b1ee9fd5a8c6e91ca6c3a11
    Port:           80/TCP
    Host Port:      0/TCP
    State:          Running
      Started:      Fri, 06 Nov 2026 11:09:31 +0000
    Ready:          True
    Restart Count:  0
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-4w66b (ro)
  nginx-two:
    Container ID:  containerd://f8470b730ed535ec2501d9e7c8f55b4dd7063ceedd096a89ec0ebcd2d543ee5b
    Image:         nginx:alpine
    Image ID:      docker.io/library/nginx@sha256:713fc2ebfd92e8217db451477182ee91944b123d2b1ee9fd5a8c6e91ca6c3a11
    Port:          8080/TCP
    Host Port:     0/TCP
    Command:
      /bin/sh
      -c
    Args:
      cat > /etc/nginx/conf.d/default.conf <<'EOF'
      server {
          listen 8080;
          location / {
              return 200 "Hello from nginx-two on port 8080";
          }
      }
      EOF
      nginx -g 'daemon off;'
```

```
    State:          Running
      Started:      Fri, 06 Nov 2026 11:09:31 +0000
    Ready:          True
    Restart Count:  0
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-4w66b (ro)
Conditions:
  Type                        Status
  PodReadyToStartContainers   True
  Initialized                 True
  Ready                       True
  ContainersReady             True
  PodScheduled                True
Volumes:
  kube-api-access-4w66b:
    Type:                    Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:  3607
    ConfigMapName:           kube-root-ca.crt
    Optional:                false
    DownwardAPI:             true
QoS Class:                   BestEffort
Node-Selectors:              <none>
Tolerations:                 node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                             node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type    Reason     Age   From               Message
  ----    ------     ----  ----               -------
  Normal  Scheduled  22s   default-scheduler  Successfully assigned default/two-container-pod to worker-02
  Normal  Pulling    22s   kubelet            Pulling image "nginx:alpine"
  Normal  Pulled     21s   kubelet            Successfully pulled image "nginx:alpine" in 1.125s (1.125s including waiting). Image size: 26368486 bytes.
  Normal  Created    21s   kubelet            Created container: nginx-one
  Normal  Started    21s   kubelet            Started container nginx-one
  Normal  Pulled     21s   kubelet            Container image "nginx:alpine" already present on machine
  Normal  Created    21s   kubelet            Created container: nginx-two
  Normal  Started    21s   kubelet            Started container nginx-two
```

```
ubuntu@master:~/kubernetes-tasks$ kubectl exec two-container-pod -c nginx-one -- wget -qO- http://127.0.0.1:8080
Hello from nginx-two on port 8080ukubectl exec two-container-pod -c nginx-two -- wget -qO- http://127.0.0.1:80 -- wget -qO- http://127.0.0.1:80
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<stytle>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
ubuntu@master:~/kubernetes-tasks$
```

5. **Set Up an Init Container in a Pod**
   Task: Modify the YAML to include an init container that sleeps for 30 seconds before the main containers start. Verify the init container's execution using kubectl describe pod and check the logs to confirm its completion.

Objective

To create a Pod containing an Init Container that executes before the main application container starts.

The Init Container should sleep for 30 seconds before completing.

Step 1: Create a YAML file

vi task5-init-container.yml

Step 2: Add the configuration

```
apiVersion: v1
kind: Pod
metadata:
  name: init-pod
spec:
  initContainers:
    - name: init-container
      image: busybox:1.36
      command: ["sh", "-c"]
      args:
        - "echo 'Init container started'; sleep 30; echo 'Init container completed'"

  containers:
    - name: nginx-container
      image: nginx:latest
      ports:
        - containerPort: 80
```

Step 3: Deploy the Pod

kubectl apply -f task5-init-container.yml

Step 4: Monitor the Pod

kubectl get pods -w

Initially, the Pod may show:

init-pod 0/1 Init:0/1

After the Init Container completes, the main Nginx container starts.

Final status:

init-pod 1/1 Running

Press Ctrl + C to stop watching the Pod status.

Step 5: Check Init Container logs

kubectl logs init-pod -c init-container

Expected output:

Init container started

Init container completed

Step 6: Verify container details

kubectl describe pod init-pod

The Init Container should show Terminated with reason Completed and exit code 0. The Nginx container should show Running.

Explanation

An Init Container is a special container that runs before the main application containers.

Important points:

- Init Containers execute sequentially.
- The main application containers start only after all Init Containers complete successfully.
- They are commonly used for initialization, configuration, and dependency checks.

```
ubuntu@master:~/kubernetes-tasks$ vi task5-init-container.yml
ubuntu@master:~/kubernetes-tasks$ kubectl apply -f task5-init-container.yml
pod/init-pod created
ubuntu@master:~/kubernetes-tasks$ kubectl get pods -w
NAME              READY   STATUS      RESTARTS      AGE
env-pod           1/1     Running     0             36m
firstpod          1/1     Running     0             38m
init-pod          0/1     Init:0/1    0             7s
nginx             1/1     Running     1 (51m ago)   21h
sleep-pod         0/1     Completed   0             32m
two-container-pod   2/2     Running     0             11m
init-pod          0/1     PodInitializing   0             31s
init-pod          1/1     Running           0             32s
^Cubuntu@master:~/kubernetes-taskskubectl logs init-pod -c init-containerer
Init container started
Init container completed
ubuntu@master:~/kubernetes-tasks$ kubectl describe pod init-pod
Name:             init-pod
Namespace:        default
Priority:         0
Service Account:  default
Node:             worker-01/172.31.0.183
Start Time:       Fri, 06 Nov 2026 11:21:01 +0000
Labels:           <none>
Annotations:      <none>
Status:           Running
IP:               10.244.1.4
IPs:
  IP:  10.244.1.4
Init Containers:
  init-container:
    Container ID:  containerd://8c095dde74c71307dac85ea61f0654bcd3760d42471b8001bc0eb5ae8d5592b4
    Image:         busybox:1.36
    Image ID:      docker.io/library/busybox@sha256:73aaf090f3d85aa34ee199857f03fa3a95c8ede2ffd4cc2cdb5b94e566b11662
    Port:          <none>
    Host Port:     <none>
    Command:
      sh
      -c
    Args:
      echo 'Init container started'; sleep 30; echo 'Init container completed'
    State:          Terminated
      Reason:       Completed
      Exit Code:    0
      Started:      Fri, 06 Nov 2026 11:21:02 +0000
      Finished:     Fri, 06 Nov 2026 11:21:32 +0000
    Ready:          True
    Restart Count:  0
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-hsnw8 (ro)
Containers:
  nginx-container:
    Container ID:   containerd://bc4d41a362bc006e4c73243095c8061e0d5c7255f6d7a8f62c506c36c58b0b4a
    Image:          nginx:latest
    Image ID:       docker.io/library/nginx@sha256:165ddde8751e75a1c907d24ef10d9f6deb08b57e0fe56caf67512486aff76d82
    Port:           80/TCP
```

```
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-hsnw8 (ro)
Containers:
  nginx-container:
    Container ID:   containerd://bc4d41a362bc006e4c73243095c8061e0d5c7255f6d7a8f62c506c36c58b0b4a
    Image:          nginx:latest
    Image ID:       docker.io/library/nginx@sha256:165ddde8751e75a1c907d24ef10d9f6deb08b57e0fe56caf67512486aff76d82
    Port:           80/TCP
    Host Port:      0/TCP
    State:          Running
      Started:      Fri, 06 Nov 2026 11:21:32 +0000
    Ready:          True
    Restart Count:  0
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-hsnw8 (ro)
Conditions:
  Type                        Status
  PodReadyToStartContainers   True
  Initialized                 True
  Ready                       True
  ContainersReady             True
  PodScheduled                True
Volumes:
  kube-api-access-hsnw8:
    Type:                    Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:  3607
    ConfigMapName:           kube-root-ca.crt
    Optional:                false
    DownwardAPI:             true
QoS Class:                   BestEffort
Node-Selectors:              <none>
Tolerations:                 node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                             node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type    Reason     Age   From               Message
  ----    ------     ----  ----               -------
  Normal  Scheduled  57s   default-scheduler  Successfully assigned default/init-pod to worker-01
  Normal  Pulling    58s   kubelet            Pulling image "busybox:1.36"
  Normal  Pulled     57s   kubelet            Successfully pulled image "busybox:1.36" in 550ms (550ms including waiting). Image size: 2217006 bytes.
  Normal  Created    57s   kubelet            Created container: init-container
  Normal  Started    57s   kubelet            Started container init-container
  Normal  Pulling    27s   kubelet            Pulling image "nginx:latest"
  Normal  Pulled     27s   kubelet            Successfully pulled image "nginx:latest" in 147ms (147ms including waiting). Image size: 63933865 bytes.
  Normal  Created    27s   kubelet            Created container: nginx-container
  Normal  Started    27s   kubelet            Started container nginx-container
ubuntu@master:~/kubernetes-tasks$
```

6. **Run a Dry Run Command to Generate YAML**
   Task: Use the kubectl run nginx --image=nginx --dry-run=client -o yaml command to generate a Pod YAML definition. Modify the generated YAML to suit specific requirements (e.g., labels or environment variables) and deploy it.

Objective

To generate a Pod YAML configuration using the kubectl run command without immediately creating the Pod, modify the generated YAML, and deploy it.

Step 1: Generate YAML using dry run

kubectl run nginx-task6 --image=nginx --dry-run=client -o yaml > task6-generated-pod.yml

This command generates the YAML file locally without creating a Kubernetes Pod.

Step 2: Open the generated YAML

vi task6-generated-pod.yml

Modify the configuration to include the required labels and environment variables.

Final configuration:

```
apiVersion: v1
kind: Pod
metadata:
 name: nginx-task6
 labels:
  app: nginx-task6
spec:
 containers:
  - name: nginx-task6
    image: nginx
    env:
     - name: myname
```

value: "sabair"

- name: City

value: "Hyderabad"

Step 3: Deploy the generated YAML

kubectl apply -f task6-generated-pod.yml

Step 4: Verify the Pod

kubectl get pods

Step 5: Verify environment variables

kubectl exec nginx-task6 -- printenv myname

Output:

sabair

kubectl exec nginx-task6 -- printenv City

Output:

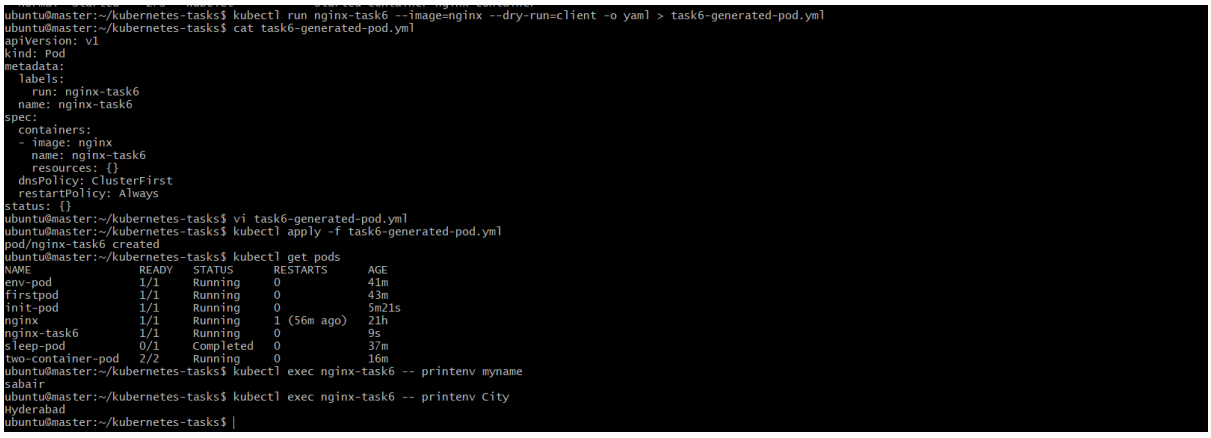
Hyderabad

Explanation

The --dry-run=client option generates or validates a resource configuration on the client side without creating the resource in the cluster.

The -o yaml option displays the generated configuration in YAML format.

This is useful when we want to generate Kubernetes YAML quickly and customize it before deployment.



7. **Use kubectl apply vs kubectl create**
   Task: Create a YAML file to define a Pod. First, deploy it using kubectl create -f <file_name>.yml and then modify the YAML (e.g., change the image version). Use kubectl apply to redeploy and verify the difference between both commands.

Objective

To understand the difference between imperative resource creation and declarative resource management using kubectl create and kubectl apply.

Step 1: Create a YAML file

vi task7-create-apply.yml

Add the following configuration:

```
apiVersion: v1
kind: Pod
metadata:
 name: create-apply-pod
 labels:
  app: create-apply
spec:
 containers:
  - name: nginx-container
    image: nginx:1.26
    ports:
     - containerPort: 80
```

Step 2: Create the Pod using kubectl create

kubectl create -f task7-create-apply.yml

Expected output:

pod/create-apply-pod created

Step 3: Try creating the same Pod again

kubectl create -f task7-create-apply.yml

Expected result:

Error from server (AlreadyExists)

This happens because the Pod already exists.

Step 4: Modify the image version

Open the YAML file:

vi task7-create-apply.yml

Change:

image: nginx:1.26

To:

image: nginx:1.27

Step 5: Apply the updated configuration

kubectl apply -f task7-create-apply.yml

Expected output:

pod/create-apply-pod configured

Kubernetes may display a warning about the missing last-applied-configuration annotation because the Pod was initially created using kubectl create without saving that configuration. This warning is not a deployment failure.

Step 6: Verify the updated image

kubectl get pod create-apply-pod -o jsonpath='{.spec.containers[0].image}'

Expected output:

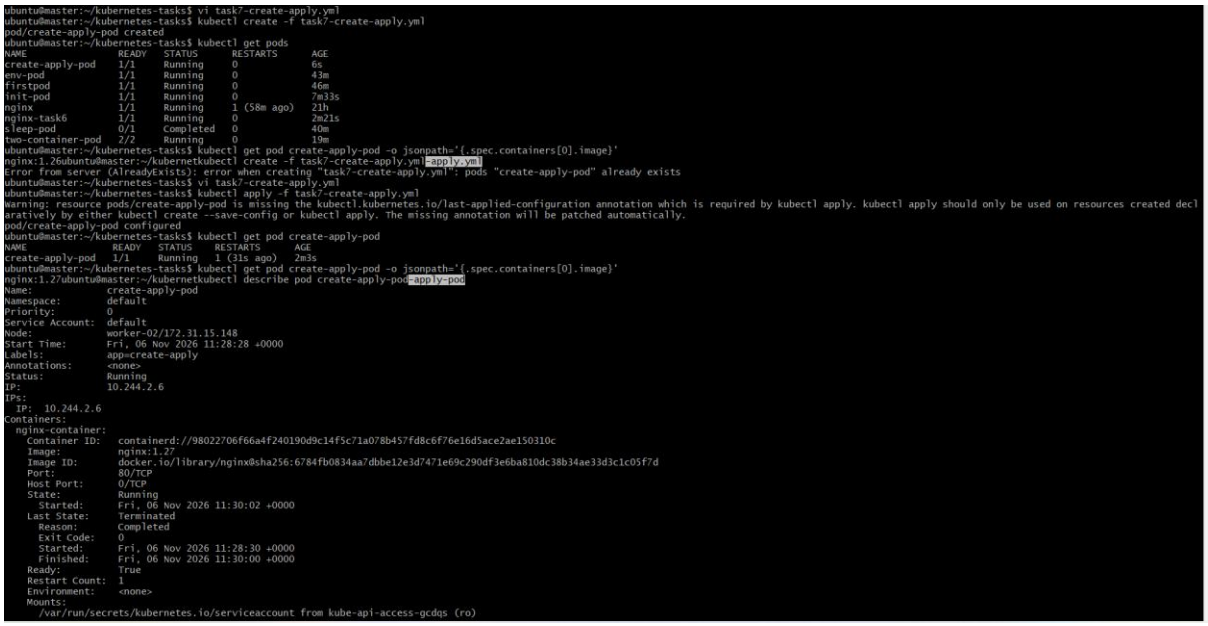
nginx:1.27

Difference Between kubectl create and kubectl apply

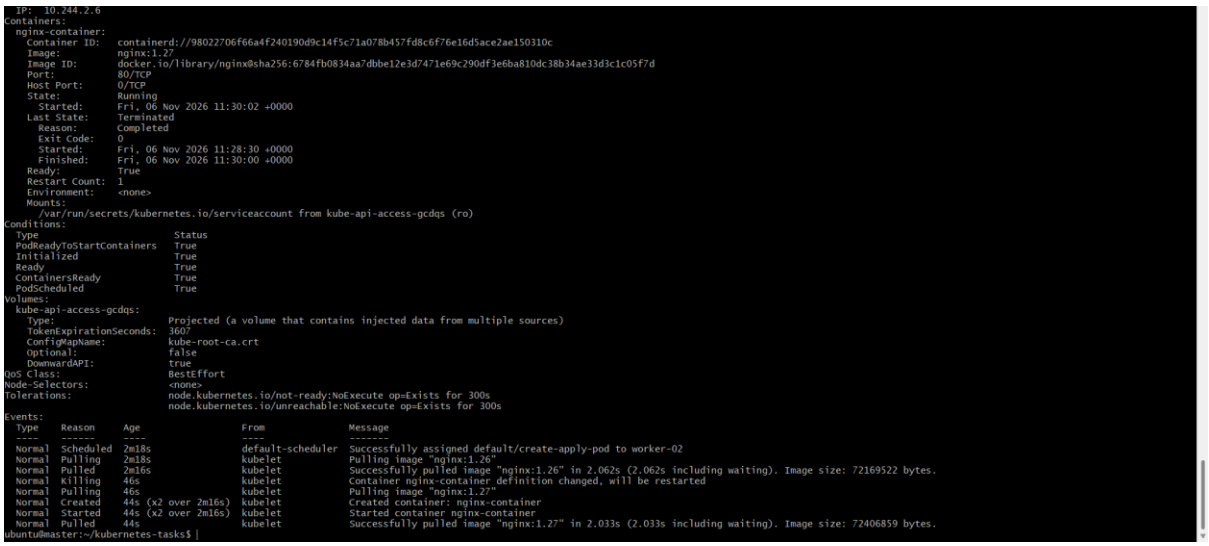
kubectl create	**kubectl apply**
Imperative command	Declarative command
Creates a new resource	Creates or updates a resource
Fails if the resource already exists	Updates an existing resource based on configuration

Commonly used for initial creation	Commonly used for ongoing configuration management

Important: Pod fields have immutability restrictions. Not every Pod change can be applied directly. For example, changing an existing Pod's environment variables generally requires recreating the Pod.



```
ubuntu@master:~/kubernetes-tasks$ vi task7-create-apply.yml
ubuntu@master:~/kubernetes-tasks$ kubectl create -f task7-create-apply.yml
pod/create-apply-pod created
ubuntu@master:~/kubernetes-tasks$ kubectl get pods
NAME               READY   STATUS      RESTARTS      AGE
create-apply-pod   1/1     Running     0             6s
env-pod            1/1     Running     0             43m
Firstpod           1/1     Running     0             46m
init-pod           1/1     Running     0             7m33s
nginx              1/1     Running     1 (58m ago)   21h
nginx-task6        1/1     Running     0             2m21s
sleep-pod          0/1     Completed   0             40m
two-container-pod  2/2     Running     0             19m
ubuntu@master:~/kubernetes-tasks$ kubectl get pod create-apply-pod -o jsonpath='{.spec.containers[0].image}'
nginx:1.26ubuntu@master:~/kubernetkubectl create -f task7-create-apply.yml-apply.yml
Error from server (AlreadyExists): error when creating "task7-create-apply.yml": pods "create-apply-pod" already exists
ubuntu@master:~/kubernetes-tasks$ vi task7-create-apply.yml
ubuntu@master:~/kubernetes-tasks$ kubectl apply -f task7-create-apply.yml
Warning: resource pods/create-apply-pod is missing the kubectl.kubernetes.io/last-applied-configuration annotation which is required by kubectl apply. kubectl apply should only be used on resources created decl
aratively by either kubectl create --save-config or kubectl apply. The missing annotation will be patched automatically.
pod/create-apply-pod configured
ubuntu@master:~/kubernetes-tasks$ kubectl get pod create-apply-pod
NAME               READY   STATUS    RESTARTS      AGE
create-apply-pod   1/1     Running   1 (31s ago)   2m3s
ubuntu@master:~/kubernetes-tasks$ kubectl get pod create-apply-pod -o jsonpath='{.spec.containers[0].image}'
nginx:1.27ubuntu@master:~/kubernetkubectl describe pod create-apply-podapply-pod
Name:             create-apply-pod
Namespace:        default
Priority:         0
Service Account:  default
Node:             worker-02/172.31.15.148
Start Time:       Fri, 06 Nov 2026 11:28:28 +0000
Labels:           app=create-apply
Annotations:      <none>
Status:           Running
IP:               10.244.2.6
IPs:
  IP:  10.244.2.6
Containers:
  nginx-container:
    Container ID:   containerd://98022706f66a4f240190d9c14f5c71a078b457fd8c6f76e16d5ace2ae150310c
    Image:          nginx:1.27
    Image ID:       docker.io/library/nginx@sha256:6784fb0834aa7dbbe12e3d7471e69c290df3e6ba810dc38b34ae33d3c1c05f7d
    Port:           80/TCP
    Host Port:      0/TCP
    State:          Running
      Started:      Fri, 06 Nov 2026 11:30:02 +0000
    Last State:     Terminated
      Reason:       Completed
      Exit Code:    0
      Started:      Fri, 06 Nov 2026 11:28:30 +0000
      Finished:     Fri, 06 Nov 2026 11:30:00 +0000
    Ready:          True
    Restart Count:  1
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-gcdqs (ro)
```



```
  IP:  10.244.2.6
Containers:
  nginx-container:
    Container ID:   containerd://98022706f66a4f240190d9c14f5c71a078b457fd8c6f76e16d5ace2ae150310c
    Image:          nginx:1.27
    Image ID:       docker.io/library/nginx@sha256:6784fb0834aa7dbbe12e3d7471e69c290df3e6ba810dc38b34ae33d3c1c05f7d
    Port:           80/TCP
    Host Port:      0/TCP
    State:          Running
      Started:      Fri, 06 Nov 2026 11:30:02 +0000
    Last State:     Terminated
      Reason:       Completed
      Exit Code:    0
      Started:      Fri, 06 Nov 2026 11:28:30 +0000
      Finished:     Fri, 06 Nov 2026 11:30:00 +0000
    Ready:          True
    Restart Count:  1
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-gcdqs (ro)
Conditions:
  Type                        Status
  PodReadyToStartContainers   True
  Initialized                 True
  Ready                       True
  ContainersReady             True
  PodScheduled                True
Volumes:
  kube-api-access-gcdqs:
    Type:                    Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:  3607
    ConfigMapName:           kube-root-ca.crt
    Optional:                false
    DownwardAPI:             true
QoS Class:                   BestEffort
Node-Selectors:              <none>
Tolerations:                 node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                             node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type    Reason     Age                From               Message
  ----    ------     ----               ----               -------
  Normal  Scheduled  2m18s              default-scheduler  Successfully assigned default/create-apply-pod to worker-02
  Normal  Pulling    2m18s              kubelet            Pulling image "nginx:1.26"
  Normal  Pulled     2m16s              kubelet            Successfully pulled image "nginx:1.26" in 2.062s (2.062s including waiting). Image size: 72169522 bytes.
  Normal  Killing    46s                kubelet            Container nginx-container definition changed, will be restarted
  Normal  Pulling    46s                kubelet            Pulling image "nginx:1.27"
  Normal  Created    44s (x2 over 2m16s)  kubelet          Created container: nginx-container
  Normal  Started    44s (x2 over 2m16s)  kubelet          Started container nginx-container
  Normal  Pulled     44s                kubelet            Successfully pulled image "nginx:1.27" in 2.033s (2.033s including waiting). Image size: 72406859 bytes.
ubuntu@master:~/kubernetes-tasks$
```

8. **Edit an Existing Pod Configuration**
   Task: Use kubectl edit pod <pod_name> to modify the running Pod's environment variables or image. After making the changes, verify if they took effect by checking the container logs or environment variables using kubectl exec.

Objective

To modify an existing Kubernetes Pod using the kubectl edit command.

In this task, we changed the Nginx container image from nginx:latest to nginx:1.27.

We used the existing env-pod created in Task 2.

Step 1: Check the existing Pod

kubectl get pods

Step 2: Check the existing image

kubectl get pod env-pod -o jsonpath='{.spec.containers[0].image}'

Initially, the output was:

nginx:latest

Step 3: Edit the Pod

kubectl edit pod env-pod

The live Pod configuration opens in an editor.

Find the image field:

image: nginx:latest

Change it to:

image: nginx:1.27

Save and exit the editor.

Step 4: Verify the updated image

kubectl get pod env-pod -o jsonpath='{.spec.containers[0].image}'

Expected output:

nginx:1.27

Step 5: Verify the Pod status

kubectl describe pod env-pod

The container should be running with the updated image.

Step 6: Verify the existing environment variables

kubectl exec env-pod -- printenv myname

Output:

sabair

kubectl exec env-pod -- printenv City

Output:

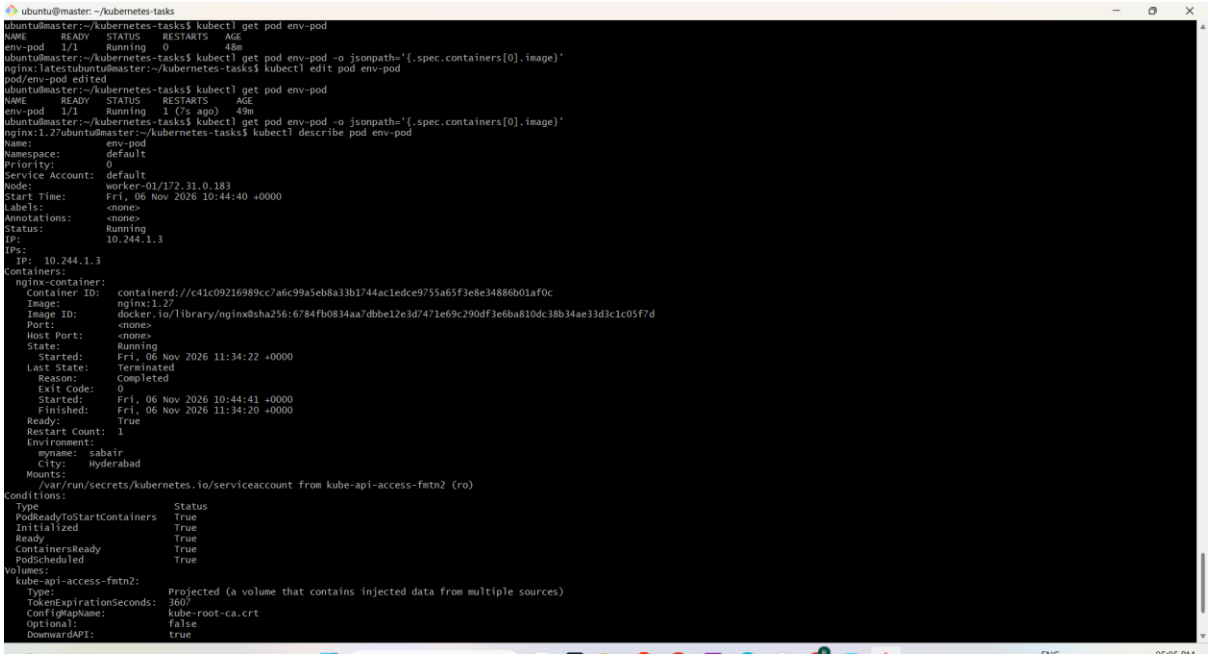
Hyderabad

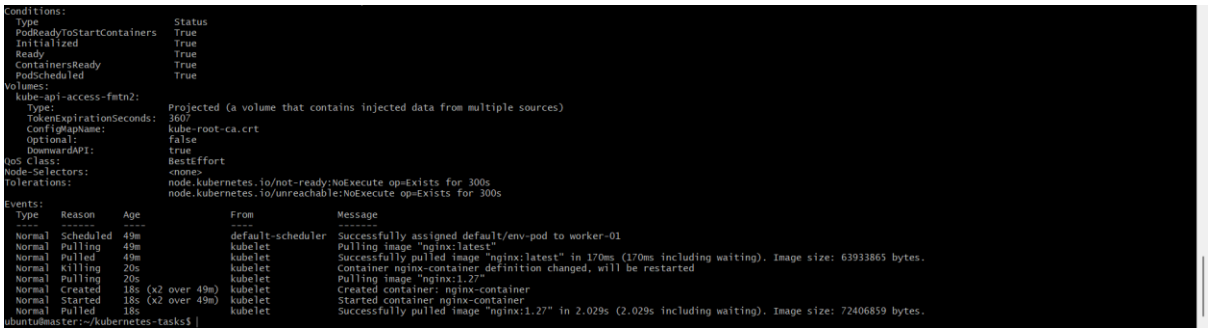
Explanation

The kubectl edit command allows us to directly edit the live configuration of an existing Kubernetes resource.

Important points:

- It opens the resource definition in an editor.
- Changes are submitted to the Kubernetes API server when saved.
- Kubernetes validates whether the requested changes are allowed.
- Not all Pod fields are editable because some fields are immutable.
- Changes made using kubectl edit do not automatically update the local YAML file.





9. **Expose a Pod Using a Service**
   Task: Create a YAML file to expose your firstpod using a Service (ClusterIP). Ensure that your service is exposing the Pod on port 80 and verify it using kubectl get svc.

Objective

To create a ClusterIP Service that allows other Pods inside the Kubernetes cluster to access the Nginx Pod.

We used the firstpod created in Task 1.

Step 1: Verify Pod labels

kubectl get pods --show-labels

The firstpod should have the label:

app=nginx

Step 2: Create a Service YAML file

vi task9-clusterip-service.yml

Step 3: Add the configuration

```
apiVersion: v1
kind: Service

metadata:
 name: firstpod-service

spec:
 type: ClusterIP

 selector:
  app: nginx

 ports:
  - protocol: TCP
    port: 80
    targetPort: 80
```

Step 4: Create the Service

kubectl apply -f task9-clusterip-service.yml

Expected output:

service/firstpod-service created

Step 5: Verify the Service

kubectl get svc

Our Service was assigned the following ClusterIP:

firstpod-service ClusterIP 10.103.145.158 <none> 80/TCP

The actual ClusterIP can differ in another cluster.

Step 6: Describe the Service

kubectl describe svc firstpod-service

Important details from our output:

Selector: app=nginx

Type: ClusterIP

IP: 10.103.145.158

Port: 80/TCP

TargetPort: 80/TCP

Endpoints: 10.244.1.2:80

Step 7: Verify Service endpoints

kubectl get endpoints firstpod-service

Our endpoint was:

10.244.1.2:80

This confirms that the Service selected the correct Pod.

Step 8: Test Service communication from another Pod

kubectl run service-test --image=busybox:1.36 --restart=Never --rm -it -- wget -qO- http://firstpod-service

Expected result:

The Nginx welcome page HTML should be displayed, followed by confirmation that the temporary test Pod was deleted.

Explanation

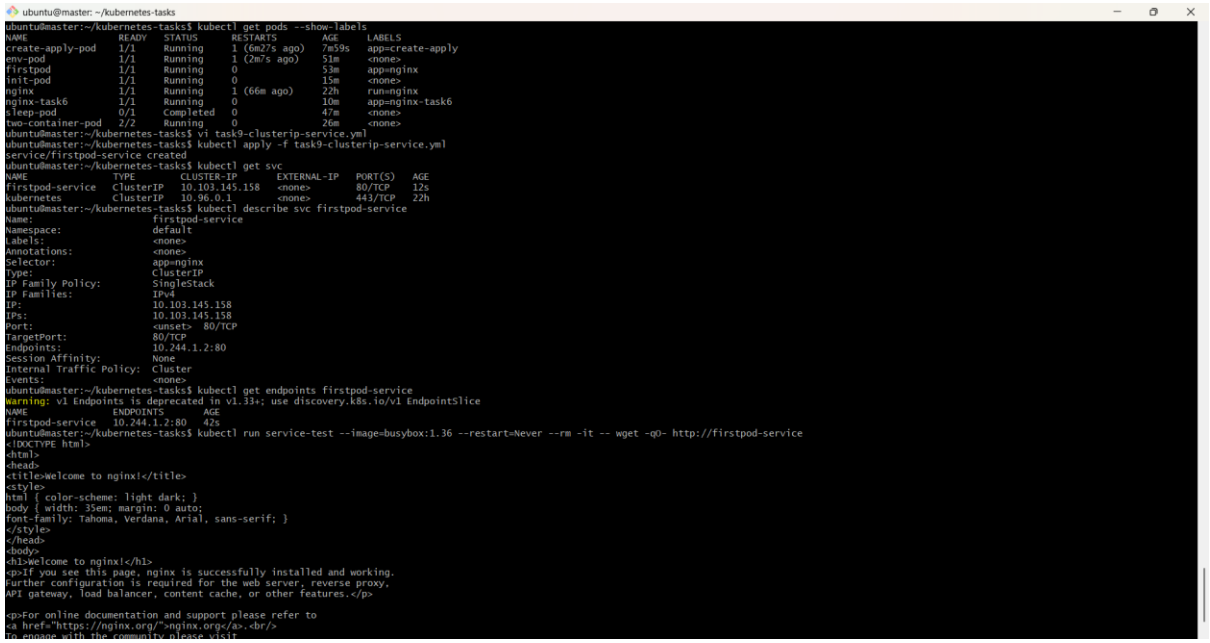
A ClusterIP Service provides an internal IP address for accessing a group of Pods.

How communication works:

1. A client Pod sends a request to the Service name.
2. Kubernetes DNS resolves the Service name to its ClusterIP.
3. The Service forwards the request to a matching Pod endpoint.
4. The application Pod processes the request and returns a response.

Important points:

- ClusterIP is the default Kubernetes Service type.
- It is accessible inside the cluster by default.
- A Service uses labels and selectors to identify backend Pods.
- The Service provides a stable access point even if individual Pod IPs change.



10. Pod with Resource Limits and Requests

Task: Add resource requests and limits to the containers in your YAML file. Specify CPU and memory requests/limits for both containers and deploy the Pod. Use kubectl describe pod to verify if the resource configurations are correctly applied.

Objective

To create a Pod containing two containers and configure CPU and memory requests and limits for both containers.

Step 1: Create a YAML file

vi task10-resource-pod.yml

Step 2: Add the configuration

```
apiVersion: v1
kind: Pod
metadata:
  name: resource-pod
spec:
  containers:
    - name: nginx-one
      image: nginx:alpine
      ports:
        - containerPort: 80
      resources:
        requests:
          cpu: "100m"
          memory: "64Mi"
        limits:
          cpu: "500m"
          memory: "128Mi"

    - name: nginx-two
      image: nginx:alpine
      command: ["/bin/sh", "-c"]
      args:
        - |
          cat > /etc/nginx/conf.d/default.conf <<'EOF'
          server {
              listen 8080;
              location / {
                  return 200 "Hello from nginx-two";
```

```
        }
      }
      EOF
      nginx -g 'daemon off;'
    ports:
     - containerPort: 8080
    resources:
     requests:
      cpu: "100m"
      memory: "64Mi"
     limits:
      cpu: "500m"
      memory: "128Mi"
```

Step 3: Deploy the Pod

kubectl apply -f task10-resource-pod.yml

Expected output:

pod/resource-pod created

Step 4: Verify the Pod

kubectl get pods

Our output showed:

resource-pod 2/2 Running

This confirms that both containers are running successfully.

Step 5: Verify CPU and memory configuration

kubectl describe pod resource-pod

For both containers, the output showed:

Limits:

 cpu: 500m

 memory: 128Mi

Requests:

cpu: 100m

memory: 64Mi

Both containers had the same resource configuration.

Step 6: Verify communication between containers

kubectl exec resource-pod -c nginx-one -- wget -qO- http://127.0.0.1:8080

Output:

Hello from nginx-two

This confirms successful communication between the two containers.

Explanation

Resource Requests

Requests specify the resources a container needs for scheduling.

The Kubernetes scheduler uses these values when deciding which worker node has sufficient available resources.

Resource Limits

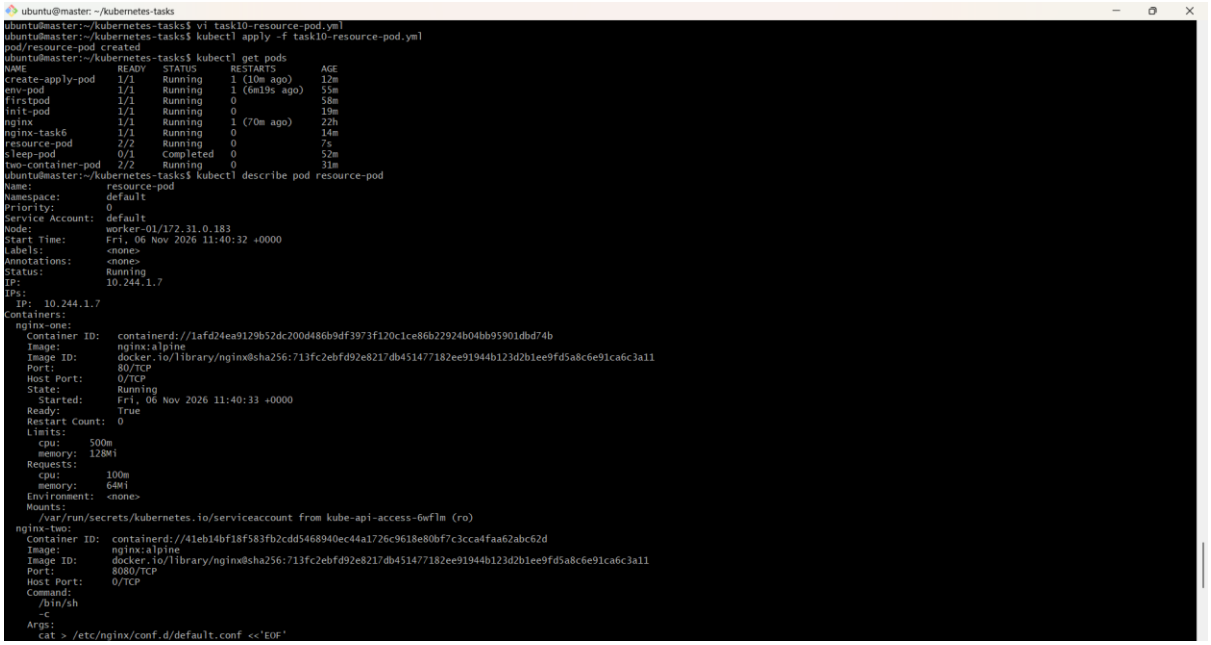
Limits define the maximum resources a container is allowed to consume.

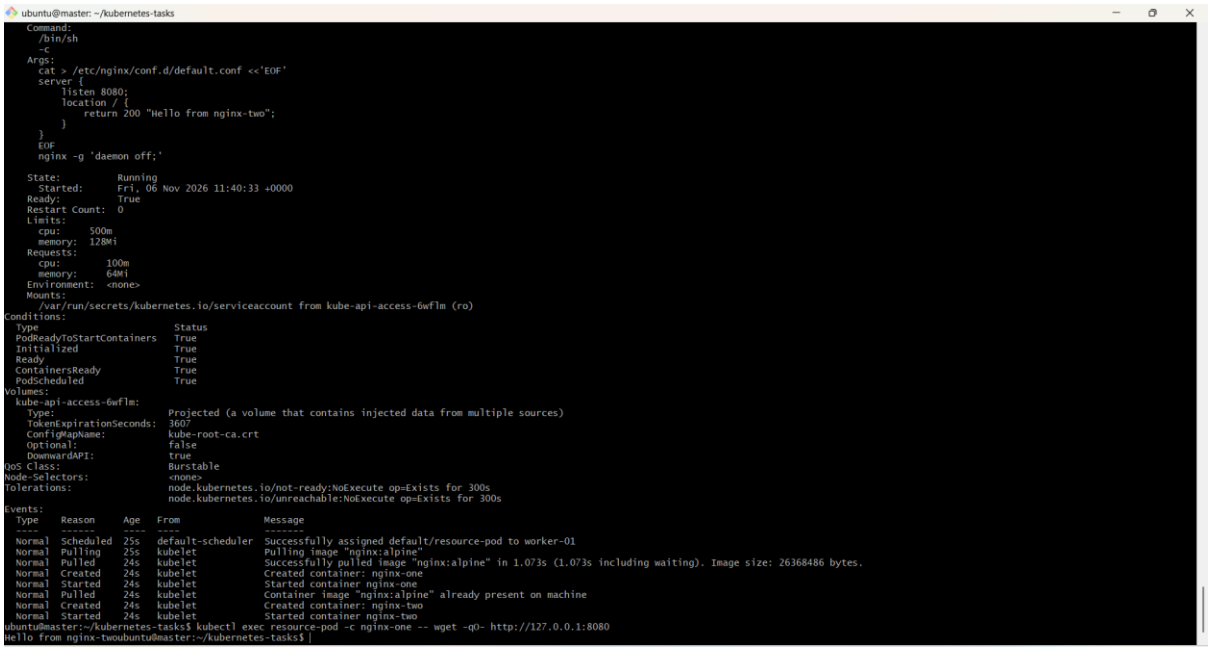
CPU and Memory Units

Unit	Meaning
100m CPU	0.1 CPU core
500m CPU	0.5 CPU core
64Mi memory	64 Mebibytes
128Mi memory	128 Mebibytes

What happens when limits are exceeded?

- CPU: The container is throttled when it tries to use more CPU than its configured limit.
- Memory: The container may be terminated with an OOMKilled reason if it exceeds its memory limit.


ubuntu@master: ~/kubernetes-tasks
ubuntu@master:~/kubernetes-tasks$ vi task10-resource-pod.yml
ubuntu@master:~/kubernetes-tasks$ kubectl apply -f task10-resource-pod.yml
pod/resource-pod created
ubuntu@master:~/kubernetes-tasks$ kubectl get pods
NAME READY STATUS RESTARTS AGE
create-apply-pod 1/1 Running 1 (10m ago) 12m
env-pod 1/1 Running 1 (6m19s ago) 55m
firstpod 1/1 Running 0 58m
init-pod 1/1 Running 0 19m
nginx 1/1 Running 1 (70m ago) 22h
nginx-task6 1/1 Running 0 14m
resource-pod 2/2 Running 0 7s
sleep-pod 0/1 Completed 0 52m
two-container-pod 2/2 Running 0 31m
ubuntu@master:~/kubernetes-tasks$ kubectl describe pod resource-pod
Name: resource-pod
Namespace: default
Priority: 0
Service Account: default
Node: worker-01/172.31.0.183
Start Time: Fri, 06 Nov 2026 11:40:32 +0000
Labels: <none>
Annotations: <none>
Status: Running
IP: 10.244.1.7
IPs:
IP: 10.244.1.7
Containers:
nginx-one:
Container ID: containerd://1afd24ea9129b52dc200d486b9df3973f120c1ce86b22924b04bb95901dbd74b
Image: nginx:alpine
Image ID: docker.io/library/nginx@sha256:713fc2ebfd92e8217db451477182ee91944b123d2b1ee9fd5a8c6e91ca6c3a11
Port: 80/TCP
Host Port: 0/TCP
State: Running
Started: Fri, 06 Nov 2026 11:40:33 +0000
Ready: True
Restart Count: 0
Limits:
cpu: 500m
memory: 128Mi
Requests:
cpu: 100m
memory: 64Mi
Environment: <none>
Mounts:
/var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-6wflm (ro)
nginx-two:
Container ID: containerd://41eb14bf18f583fb2cdd5468940ec44a1726c9618e80bf7c3cca4faa62abc62d
Image: nginx:alpine
Image ID: docker.io/library/nginx@sha256:713fc2ebfd92e8217db451477182ee91944b123d2b1ee9fd5a8c6e91ca6c3a11
Port: 8080/TCP
Host Port: 0/TCP
Command:
/bin/sh
-c
Args:
cat > /etc/nginx/conf.d/default.conf <<'EOF'


ubuntu@master: ~/kubernetes-tasks
Command:
/bin/sh
-c
Args:
cat > /etc/nginx/conf.d/default.conf <<'EOF'
server {
listen 8080;
location / {
return 200 "Hello from nginx-two";
}
}
EOF
nginx -g 'daemon off;'
State: Running
Started: Fri, 06 Nov 2026 11:40:33 +0000
Ready: True
Restart Count: 0
Limits:
cpu: 500m
memory: 128Mi
Requests:
cpu: 100m
memory: 64Mi
Environment: <none>
Mounts:
/var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-6wflm (ro)
Conditions:
Type Status
PodReadyToStartContainers True
Initialized True
Ready True
ContainersReady True
PodScheduled True
Volumes:
kube-api-access-6wflm:
Type: Projected (a volume that contains injected data from multiple sources)
TokenExpirationSeconds: 3607
ConfigMapName: kube-root-ca.crt
Optional: false
DownwardAPI: true
QoS Class: Burstable
Node-Selectors: <none>
Tolerations: node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
Type Reason Age From Message
Normal Scheduled 25s default-scheduler Successfully assigned default/resource-pod to worker-01
Normal Pulling 25s kubelet Pulling image "nginx:alpine"
Normal Pulled 24s kubelet Successfully pulled image "nginx:alpine" in 1.073s (1.073s including waiting). Image size: 26368486 bytes.
Normal Created 24s kubelet Created container: nginx-one
Normal Started 24s kubelet Started container nginx-one
Normal Pulled 24s kubelet Container image "nginx:alpine" already present on machine
Normal Created 24s kubelet Created container: nginx-two
Normal Started 24s kubelet Started container nginx-two
ubuntu@master:~/kubernetes-tasks$ kubectl exec resource-pod -c nginx-one -- wget -qO- http://127.0.0.1:8080
Hello from nginx-twoubuntu@master:~/kubernetes-tasks$